

Synera: Synergy Auto-Arena

高级程序设计课程 PA 指导

2026 年 5 月 8 日

摘要

自走棋是近年来流行的策略游戏品类, 代表作有《The Last Flame》和《Teamfight Tactics (云顶之弈)》。其核心玩法分为三个阶段: 玩家在准备阶段用金币招募英雄并进行排兵布阵, 战斗阶段中英雄自动寻路攻击敌人并释放技能, 结束后根据胜负结算金币。

本次 PA 名为 **Synera: Synergy Auto-Arena**, 便是用 C++ 实现一个单机版轻量级自走棋游戏 (PVE)。Let's Go!

0 开始之前

WARNING

All of the code for this project must be your own. You may not copy source code from other students or other sources that you find on the web. Plagiarism will not be tolerated.

1. 我们不禁止 (但是也不推荐) 在开源项目基础之上完成该项目, 但是在验收的时候需要做到两点: 主动告知使用的开源项目情况; 强调你所做的修改, 尤其是和本课程相关的修改。
2. 举一个极端例子, 假如一个开源项目已经达到我们的需求, 但是如果你重构了它的代码, 以更符合面向对象思想实现, 那么不会认为是抄袭; 但是如果直接拿这个项目验收而不修改 (或者修改很少), 那么将会判定为抄袭。注意: 图像素材的替换不会被认为和本课程相关。
3. 本次 PA 共分为四个阶段, 每个阶段为期两周, 共需约八周时间完成。验收预计将于第 6 周和第 8 周进行, 分别检查基础功能与扩展功能。具体的实现顺序不强制要求, 但验收时将严格按照本文档的功能要求进行检查。

形式化说明

为统一实现语言，建议采用如下记号（允许在代码中做等价替换）：

- **棋盘与地块**：主棋盘记为 $B = [1, M] \times [1, N]$ ，地块坐标记为 (x, y) 。占用函数 $\text{occ}_r(x, y) \in \mathcal{U}_r \cup \{\emptyset\}$ 表示第 r 轮该格上的单位。
- **备战区**：备战区索引集合记为 $\mathcal{K} = \{1, \dots, K\}$ （如 $K = 8$ ），每个槽位至多容纳一个单位。
- **玩家与单位**：玩家对象记为 p ，仅表示经营/决策主体；单位对象记为 $u \in \mathcal{U}$ 。我方与敌方单位同构，均由 `Unit` 表示。
- **控制归属与羁绊标签**：定义 $\text{owner}(u) \in \{\text{PlayerCtrl}, \text{EnemyCtrl}\}$ 区分我方/敌方；定义 $\text{traits}(u) \subseteq \mathcal{T}$ 表示职业/种族等羁绊标签（用于羁绊计算，不用于敌我区分）。
- **轮次与阶段**：第 r 轮单位集合为 $\mathcal{U}_r = \mathcal{A}_r \cup \mathcal{E}_r$ ，其中 $\mathcal{A}_r$ 为我方参战集合， $\mathcal{E}_r$ 为敌方生成集合。流程阶段记为 `Prep`→`Combat`→`Resolve`。
- **战斗状态机**：单位状态集合 $\mathcal{S} = \{\text{Idle}, \text{Moving}, \text{Attacking}, \text{Casting}, \text{Dead}\}$ 。
- **索敌与攻击**：对存活单位 u ，候选敌人集合定义为

$$\text{Opp}_r(u) = \{v \in \mathcal{U}_r \mid \text{owner}(v) \neq \text{owner}(u), v \text{ 存活}\}.$$

目标选择先最小化欧氏距离，再按预设平局规则（如生命值、坐标序）处理。

- **经济与人口**：金币记为 g ；购买/刷新/升级人口对应状态转移 $g \leftarrow g - c$ 。上阵约束为 $|\mathcal{A}_r| \leq \text{cap}(p)$ 。
- **羁绊**：对标签 $t \in \mathcal{T}$ ，计数函数

$$n_t = |\{u \in \mathcal{A}_r \mid t \in \text{traits}(u)\}|,$$

当 n_t 达到阈值集合 Θ_t 时激活对应效果。

- **升星与装备**：若存在同名同星单位三元组则触发合并算子 `merge3(·)`；装备槽函数 `eq(u)` 满足容量约束（如 1 件）。
- **存档**：存档状态可简记为（在具体实现中应当由你自行修改）

$$\Sigma = (p, r, \text{phase}, \mathcal{A}_r, \mathcal{E}_r, \text{Bench}, \text{Shop}, \text{EquipPool}).$$

读档即恢复 Σ 。

1 前言

自走棋 (Auto-battler) 是近年来极具策略深度的游戏品类，代表作有《The Last Flame》和《Teamfight Tactics (云顶之弈)》。在本次项目中，我们将实现一个名为 **Synera: Synergy Auto-Arena** 的单机版轻量级自走棋游戏。

在 Synera 中，玩家可以进行招募单位、排兵布阵、组合羁绊并观看单位与 AI 敌人自动战斗 (PVE) 等操作。该项目主要考察面向对象设计 (OOP)，同时会涉及一些简单的状态机管理、寻路算法以及图形界面编程能力。项目代码规模预计在 **3000 行以上**。

分数构成与验收说明

本项目分为四个阶段。验收将进行两次：

- 第一次验收：在阶段三结束后进行，检查阶段一至阶段三的所有基础功能。
- 第二次验收：在阶段四结束后进行，检查扩展功能。

评分项	占比
各阶段基础功能	70% (阶段一、二、三的基础功能实现)
扩展内容	10% (根据阶段四实现的实际情况给分)
代码风格	10% (OOP 运用、代码风格与规范)
文档与提交规范	10% (包含 README / AI 使用说明)

2 阶段一

2.1 棋盘与备战区

游戏的战场在逻辑上是一个 $M \times N$ 的网格 (例如 8×8)。网格分为玩家半场 (下半区) 和敌人半场 (上半区)。每个 1×1 的矩形是一个地块，每个地块同一时间只能容纳一个单位。

除了主棋盘外，玩家还拥有一维的**备战区** (Bench，例如 8 个格子)，用于存放已购买但尚未上阵的单位。

2.2 基础单位与交互

单位是游戏中的核心实体。每个单位拥有基础属性：生命值 (HP)、攻击力 (ATK)、攻击距离 (Range)、最大法力值 (Max Mana) 和当前法力值 (Mana)。

你可以基于此设计基类 `Unit` 及其派生类，对于各种派生类而言你可以涉及可以有一些更多的属性，如装备、buff、技能、星阶、控制归属、阵营、羁绊等 (当然你也可以

在 `Unit` 基类中设置这些字段并初始化为空，这取决于你自己的设计)。

对象模型说明

为避免概念混淆，逻辑层面约定如下：

- **玩家 (Player)**: 玩家是全局控制实体，记为 p 。玩家拥有资源与进度状态（血量、金币、等级、人口上限、当前关卡等），并在准备阶段对己方单位进行经营与布阵决策。
- **单位 (Unit)**: 单位是战斗实体，记为 $u \in \mathcal{U}$ 。每个单位采用统一的数据结构（如 HP、ATK、Range、Max Mana、Mana、位置、状态等）。
- **敌我归属**: 定义函数

$$\text{owner} : \mathcal{U} \rightarrow \{\text{PlayerCtrl}, \text{EnemyCtrl}\}.$$

“我方单位/敌方单位”仅表示 $\text{owner}(u)$ 取值不同，本质上是同一类 `Unit`，不应建成两套不同类型体系。

- **羁绊标签**: 定义 $\text{traits}(u) \subseteq \mathcal{T}$ ，用于职业/种族羁绊的计数与效果激活，不用于敌我区分。
- **轮次单位集合**: 第 r 轮战斗中，单位集合定义为

$$\mathcal{U}_r = \mathcal{A}_r \cup \mathcal{E}_r, \quad \mathcal{A}_r \cap \mathcal{E}_r = \emptyset,$$

其中 $\mathcal{A}_r$ 为我方参战单位集合， $\mathcal{E}_r$ 为该轮敌方单位集合。

- **敌对关系**: 对任意存活单位 u ，其可选敌方目标集合定义为

$$\text{Opp}_r(u) = \{v \in \mathcal{U}_r \mid \text{owner}(v) \neq \text{owner}(u), v \text{ 存活}\}.$$

玩家是游戏的全局实体，具有血量、金币、等级、已有单位、人口上限等属性。玩家血量归零则游戏失败。

单位系统采用统一类型建模：我方单位与敌方单位均为 `Unit` 实例，通过 `owner` 字段区分敌我控制归属；`traits` 字段用于羁绊系统。第 r 轮开始时，系统按关卡配置生成敌方集合 $\mathcal{E}_r$ 并部署于敌方半场；我方参战集合记为 $\mathcal{A}_r$ 。若 $\mathcal{E}_r$ 全灭则本轮胜利，若 $\mathcal{A}_r$ 全灭则本轮失败并扣除玩家一定血量。轮次结算后清空本轮战斗集合并进入下一轮。

在阶段一，玩家需要能够使用鼠标在备战区和玩家半场之间**自由拖拽**单位，调整站位。如果拖拽的目标地块已被占用，应有合理的非法放置处理逻辑（例如交换位置或弹回原位）。

2.3 GUI

游戏需要一个图形界面 (GUI)。阶段一的 GUI 需要能展示你的战斗棋盘、备战区网格，以及上面的单位。你还需要在界面上直观地展示单位的生命条、法力条或基础属性面板。

界面的精美程度不影响分数，但必须清晰表达逻辑。

阶段一 Checklist

- [] 实现 $M \times N$ 棋盘、玩家半场/敌方半场与地块占用规则。
- [] 实现备战区 (Bench) 与棋盘之间的数据同步。
- [] 实现 Unit 基类 (HP/ATK/Range/Max Mana/Mana 等字段)。
- [] 我方/敌方采用统一 Unit 类型实现，仅用 owner 区分控制归属，traits 仅用于羁绊。
- [] 实现玩家实体与敌方轮次生成 ($\mathcal{E}_r$)。
- [] 实现拖拽摆放与非法放置处理 (交换或回弹)。
- [] GUI 展示棋盘/备战区/单位信息 (血条、蓝条或属性面板)。

3 阶段二

3.1 战斗流程与状态机

在这个阶段，你需要实现单机 PvE 模式：在战斗阶段开始时，敌人半场会根据当前关卡预设，自动生成敌人阵容。

游戏分为**准备阶段**（排兵布阵），**战斗阶段**（不可移动，由 AI 接管）与**结算阶段**，三个阶段作为一次 Loop，每一轮会更新遇到的敌人直至游戏结束。

- 准备阶段，可以从商店购买单位，更换装备，排兵布阵，调整上场单位，升级人口数等。
- 战斗开始时，敌人半场根据关卡自动生成敌人阵容。建议为单位实现有限状态机 (如 Idle, Moving, Attacking, Casting, Dead) 以管理战斗逻辑。
- 结算阶段，若此轮胜利，敌人有概率掉落装备，且玩家获得较高的金币数；若此轮失败，则扣除玩家一定血量，此时玩家获得较低的金币数。

- **胜负条件**：玩家血量归零时游戏失败；击败所有预设关卡（如 3 轮）后游戏胜利。你也可以选择无限模式，敌人随轮次递增难度。
- **关卡推进**：每轮敌人阵容由当前关卡数决定，随关卡推进敌人数量、属性和种类逐步增强（如第 1 轮仅 1-2 个弱小单位，第 3 轮出现多个高属性精英）。

3.2 寻路与移动

战斗中，单位自动搜索并锁定**欧氏距离**最近的敌方目标。

- 目标在**攻击距离内**则站桩攻击；否则向目标移动。
- **碰撞与阻挡**：移动时不能穿过任何单位，必须进行绕行。需实现基础寻路与防重叠碰撞机制（即在移动过程中同一地块只能容纳最多一个单位）。
- 如遇到距离相同的两个敌方目标，请按照优先生命值高低、从左向右、从下到上的方式索敌。
- **敌方单位**也具有攻击范围/攻击目标，也可能进行移动，向着它的目标移动。

3.3 攻击与技能

- **自动攻击**：单位根据一定的攻击速度，周期性对目标造成伤害。每次普攻回复一定法力值。
- **多态技能**：当法力值满时，单位自动释放技能（法力值清零）。你必须利用面向对象的多态特性，实现 **3-5 种**不同的英雄单位及其独特的技能效果（例如：单体眩晕、直线 AOE 伤害、友军范围回血等）。
- 一个单位生命值 ≤ 0 时死亡。当**一方所有单位生命值全部** ≤ 0 时战斗结束，结算胜负。

参考数值（可自行调整）

单位初始法力值	0（或各英雄自定义初始法力）
法力获取	每次普攻回复 10 点法力值
最大法力值	例如 60（各英雄可不同）
攻击速度	以帧数计量，如每 60 帧（约 1 秒）攻击一次
移动速度	以帧数计量，如每 20 帧移动一格
伤害数值	1 星单位 ATK 建议 20-50，HP 建议 200-500
2 星单位属性	建议为 1 星的 1.5-2 倍
敌方随关卡数增长	每 3 轮敌人属性提升约 20%

阶段二 Checklist

- [] 完成准备/战斗/结算三阶段循环与轮次推进。
- [] 敌方单位按关卡配置自动生成并随轮次增强。
- [] 实现单位状态机 (Idle/Moving/Attacking/Casting/Dead)。
- [] 实现索敌规则 (欧氏距离 + 平局优先级)。
- [] 实现寻路、阻挡与防重叠碰撞。
- [] 实现普攻、回蓝、技能多态 (3-5 英雄) 与胜负结算。

4 阶段三

4.1 商店与经济

引入金币系统。每完成一个关卡，玩家获得固定的基础金币。

游戏界面需包含一个**招募区 (Shop)**，提供 5 个随机刷新的单位供玩家购买。玩家扣除相应金币后，购买的单位会出现在备战区。玩家也可花费少量金币手动刷新商店。

人口系统：场上单位数受人口上限约束，初始人口上限为 3-4。玩家可花费金币升级人口上限（如每级花费递增），每提升一级人口上限 +1。当场上单位数已达人口上限时，不能再从备战区拖拽单位上阵。备战区不受人口限制（比如固定为 8）。

4.2 羁绊系统

你需要设计并实现 **4-6 种职业/种族（羁绊）**。多名不同同职英雄在场时激活全局 Buff（如属性光环、机制改变）。每个英雄至少拥有 1-2 种标签。

半开放式要求：你可以自行命名和设计这些羁绊，但至少包含类似以下机制的实现：

- **属性光环类**：如”战士”达到 2/4 个时，所有战士增加护甲/生命值。
- **机制改变类**：如”法师”达到 3 个时，所有友军技能伤害翻倍；或”游侠”有概率连续攻击两次。

最低要求：在 4-6 种羁绊中，至少包含 **2 种属性光环类**和 **1 种机制改变类**，其余可自由设计。每种羁绊需有明确的触发人数阈值和效果描述。

4.3 进阶与装备

你需要设计升星与装备掉落机制，至少要包含如下要求：

升星机制：当玩家购买或获得单位后，若备战区与场上合计拥有 **3 个完全相同的 1 星单位**（即同名同星级），则在**准备阶段自动合并**为 1 个属性提升的 **2 星单位**（合并发生在最近获得第 3 个的时刻，合并后该单位保留在原位）。2 星单位的属性应获得显著提升（建议为 1 星的 1.5-2 倍）。

装备掉落：击败敌人后有概率掉落基础装备，装备出现在备战区旁的装备栏中。玩家可将装备拖拽至单位身上穿戴。每个单位最多穿戴 **1 件**装备（2 星单位可扩展为 2 件，作为可选扩展）。你必须至少实现以下 **4 种**基础装备：

装备名称	效果
铁剑	攻击力 +15
锁子甲	生命值 +150
急速手套	攻击速度提升 20%
蓝水晶	最大法力值 -30（使技能更快释放）

你也可以在此基础上自由添加更多装备种类。

4.4 游戏存档

游戏需要支持存档功能。你需要自行实现一套存档与读档功能（序列化与反序列化），保存当前游戏的金币、关卡进度、场上与备战区的单位状态、穿戴的装备等，并写入到文件中。保存好的存档应能被重新加载，完全恢复到存档时的状态。

在第一次验收时，将全面检查阶段一至阶段三的各项功能与存档读取机制。

4.5 GUI

当前阶段需完成整个游戏的 GUI 设计，需展示你的战斗棋盘、备战区网格、上面的单位、玩家血量/金币数/人口数、商店招募区、装备/羁绊/星级显示、当前轮次/阶段/准备时间等，在界面上直观地展示单位的生命条、法力条或基础属性面板。

界面的精美程度不影响分数，但必须清晰表达逻辑。

阶段三 Checklist

- [] 实现金币系统、关卡奖励与商店（5 个招募位）。
- [] 实现购买、刷新与备战区落位逻辑。
- [] 实现人口上限升级与上阵人数限制。
- [] 实现 4–6 种职业/种族（至少出现 2 种属性光环 + 1 种机制改变类型的羁绊）。
- [] 实现升星（3 合 1）与 2 星属性提升。
- [] 实现装备掉落、穿戴限制与至少 4 种基础装备。
- [] 实现存档/读档机制。
- [] GUI 完整展示经济、商店、羁绊、星级、轮次与阶段信息。

5 阶段四

此阶段为扩展阶段，占总分的 10%。你可以选择实现以下推荐的扩展功能，也可以加入任何你觉得很酷的想法。只要实现了一个具备相当工作量的合理拓展功能，就能得到扩展部分的满分。推荐方向：

- **高级经济系统**：引入每 10 金币产生 1 金币的”利息机制”，以及连胜/连败的额外金币奖励。
- **装备合成树**：引入装备合成系统。两件特定基础装备穿戴在同一单位身上时，自动合成为一件具有特殊被动效果的”高级装备”（如复活甲、吸血剑）。
- **视听结合**：添加背景音乐、攻击音效。为远程攻击添加弹道飞行物（Projectiles），为技能添加范围特效等。
- **局域网联机**：将 PvE 改为真正的 PvP。使用 Socket 实现局域网联机，两名玩家各自运营，战斗阶段互相将镜像军队发送给对方进行自动对战。
- **战争迷雾/视野系统**：增加特殊模式，部分棋盘被遮挡。
- **智能索敌**：根据敌方每个单位对我方的威胁程度进行索敌。

阶段四 Checklist

- [] 选择并实现至少 1 个具备实质工作量的扩展功能。
- [] 扩展功能与现有系统正确集成（流程可正常闭环）。
- [] 完成对应 GUI/交互或系统逻辑展示。
- [] 在 README 中补充扩展设计、实现说明与演示要点。

6 实验指南

实验指南旨在降低同学们进行实验的难度，里面的内容不是强制要求，希望能给你带来帮助。

6.1 GUI 与 Starter Code

往年的同学大多使用 Qt 来完成实验，但本项目不限制于 Qt，SDL2 等第三方库也是允许的，界面好坏不影响基础分数，但必须使用 C/C++ 开发。

图形库推荐

1. **Qt**: (<https://www.qt.io/>) 是非常流行的 C++ 图形应用开发框架。体量较大，功能较多。网络上有较多的 Qt 教学资源。这里给出两份 Qt 教程：<https://qtguide.ustclug.org/> 贴合 QtCreator 软件，从最基础的底层原理开始教学，深入浅出；<https://wizardforcel.gitbooks.io/qt-beginning/content/> 从实际开发 App 的角度开始，方便快速上手。
2. **SDL2**: (<https://www.libsdl.org/>) 是相对流行的基于 C 的轻量级图形渲染库。提供了基本的绘图功能（如刷新屏幕，渲染图片等）。体积很小，功能比较基础，但是易于学习。这是一份 SDL2 教程：<https://lazyfoo.net/tutorials/SDL/>。

为了帮助大家快速上手 QT 的可视化，**我们提供了一份基础的 Starter Code**。该 Demo 使用 Qt 实现了一个简易的网格框架，同时包含了基础的网格、拖拽机制。**你可以基于此 Demo 深入开发/重构 OOP 逻辑，或者将其作为参考，从零构建你自己的 PA proj。**

6.2 基于帧的逻辑与状态机

在自走棋中，多个单位同时进行移动、攻击和施法。处理这种并发逻辑的最佳方式是使用**基于帧的时间循环**（如设定 60 fps，每帧调用所有单位的 `update()` 方法）。

同时，强烈建议为每个单位实现一个**有限状态机 (FSM)**。一个单位在任何时刻应当处于以下状态之一：`Idle`（空闲）、`Moving`（移动中）、`Attacking`（攻击前摇）、`Casting`（施法中）、`Dead`（死亡）。清晰的状态划分能极大减少 Bug 的产生。

6.3 寻路与可达性

寻路是本项目的核心难点之一。在二维网格（`TileMap`）中，你可以使用广度优先搜索（BFS）或 A^* 算法。

由于战场上充满了不断移动的其他单位，目标单位也可能在移动，因此寻路**不应该是静态的**。你可能需要让单位每走几步或目标位置发生变化时，重新计算一次路径（Re-pathing）。同时要处理“多个近战单位围殴一个敌人，导致无路可走”的碰撞阻塞情况。

6.4 任务优先级与目标锁定

一个单位该攻击谁？通常是欧氏距离最近的敌人。如果存在多个距离相同的敌人，可以通过特定的优先级规则（如优先攻击当前血量最低的）来打破僵局。当目标死亡后，单位需要立即回到 `Idle` 状态并重新搜索新的目标。

6.5 素材获取

在本项目中，素材主要包括：角色（Unit）、技能特效、UI 界面、音效等。良好的素材可以提升展示效果，但不会影响基础分数，建议优先保证功能实现。

推荐使用以下免费资源：

- **Kenney** (<https://kenney.nl/assets>)：推荐搜索“strategy”，“RPG”，“UI”分类。其中 1-bit pack、Tiny RPG Forest 等素材包非常适合网格游戏。
- **OpenGameArt** (<https://opengameart.org>)：搜索“character sprite 32x32”或“tilemap RPG”可找到大量角色和地图素材。注意筛选 License（优先选择 CC0 或 CC-BY）。
- **itch.io** (<https://itch.io/game-assets>)：筛选标签“sprite”，“characters”，“tiles”，可找到大量免费像素风格素材包。
- **CraftPix** (<https://craftpix.net>)：提供大量高质量 2D 游戏素材（角色、场景、UI 等），可直接搜索“2D character”，“fantasy RPG”，“top-down”等关键词。

素材使用建议：

- **角色图标**：建议使用 32×32 或 64×64 的 PNG 精灵图。如果找不到合适的图片，用带颜色区分的纯色方块 + 文字标注（如红色方块写”战士”、蓝色写”法师”）完全足够，不影响评分。
- **棋盘与 UI**：简单的色块填充即可，无需精致贴图。如果实在找不到素材，通过 Qt 可直接使用 QPainter 绘制矩形和文字。
- **格式**：优先使用 PNG（支持透明通道），避免 JPG（无透明背景）。
- **素材管理**：建议在项目目录下创建 assets/ 或 res/ 文件夹，按 assets/units/、assets/ui/、assets/effects/ 分类存放。

WARNING

在素材选择上，请优先选择可免费使用（如 CC0）的素材；如需署名，请在项目中注明来源；不建议直接使用商业游戏素材（如金铲铲之战）。

本课程重点并不在于 GUI 设计，你甚至可以使用简单图形作图，整体游戏的 GUI 精美与否并不会影响多少实验的整体分数。

7 如何提问

Question 7.1

如何正确地向助教提问？

向助教提问时，请遵循以下原则：

- **提供足够上下文**：说明你的问题出现在哪个模块、你尝试了哪些方法、遇到的具体错误信息。
- **先搜索/AI 再提问**：请先通过搜索引擎（Google、Stack Overflow 等）查找类似问题，或向 Chatgpt、Deepseek 等大模型提问。如果能轻松找到答案，说明你没有尝试自己解决问题。
- **格式化代码片段**：如果问题涉及代码，请使用 markdown 代码块格式化，方便阅读。
- **单次提问聚焦一个问题**：不要在一个问题中包含多个不相关的疑问，这会增加回答的复杂度。

Question 7.2

哪些问题是不适合向助教提问的？

- **不相干的问题**：如“我的电脑出故障了”、“文件找不到了”等，这类问题...
- **编译 error**：编译器会明确指出错误原因，请仔细阅读错误提示并修正。
- **未经搜索的问题**：能在前 3 个搜索结果中找到答案的问题，不要直接拿来问。

如果遇到无法自行解决的编程难题，欢迎随时联系助教，但请务必提供：你的运行环境、具体的错误信息或异常堆栈、你已经尝试过的解决方法等信息。

8 项目提交与文档说明

8.1 提交格式

最终提交内容为**整个项目压缩包**（.zip 或 .tar.gz），请务必在打包前剔除编译生成的缓存文件夹（如 build/、*.o），只保留源码、资源、文档和游戏**最终的可执行文件**。

8.2 项目文档 (README)

根目录下需提供 README.md，包含以下内容：

1. **基本信息**：姓名，学号，项目各阶段完成度。
2. **文件树结构**：展示目录树并简要解释核心文件夹。
3. **核心类和数据结构**：列出自行创建的核心类（如 Unit, Board）及其主要功能。
4. **算法描述**：简述关键算法（寻路、目标锁定、羁绊计算）。
5. **辅助函数**：简明扼要地说明全局工具函数的作用（如果你用到了一些辅助函数）。

8.3 AI 使用说明

利用 AI 辅助是被鼓励的，但必须**完全掌控代码**。文档中须单设一节”AI 使用说明”：

- **项目规划**：说明如何构思并将项目拆分交由 AI 辅助。
- **核心代码解析**：挑出至少**两个**由 AI 辅助生成的核心模块，用自己的话详细解释其运作原理。若验收时发现代码存在复杂的 AI 生成痕迹却无法解释，将面临严重扣分。

9 代码风格

- **面向对象 (OOP)**: 核心实体 (单位、技能) 必须使用继承和多态实现, 构建基类并派生具体英雄类。
- **泛型编程**: 鼓励使用 STL (如 `std::vector`, `std::map`) 管理单位和羁绊。
- **异常处理**: 存档读档时推荐使用 `try-catch` 防止崩溃, 确保游戏在出现常规错误时不会发生 Segfault (如在验收阶段发生异常, 将酌情扣分)。
- **代码格式化**: 时刻保持代码整洁, 缩进与命名风格一致, 推荐使用 `clang-format`。

10 说明与致谢

10.1 文档说明

- 本文档旨在提供项目实现的统一参考与验收边界说明, 帮助同学明确阶段目标与最低要求。
- 文档中的示例数值、实现细节与技术路线均为建议, 不限制你在满足功能要求前提下进行合理扩展。
- 若本文档与后续课程公告存在冲突, 以课程主页、课堂通知或助教统一说明为准。

10.2 项目说明

- 本项目的玩法灵感主要来源于 Auto-battler 类型游戏及其常见设计范式, 如回合制运营、自动战斗、单位成长与羁绊构筑等。
- 课程项目在此基础上进行了教学化抽象, 重点服务于面向对象设计能力的训练。

10.3 致谢

- 感谢每一位同学参与高级程序设计课程 PA 项目, 在实践中投入时间与热情。
- 感谢同学们在开发过程中的交流与反馈, 这些讨论将推动 PA 与课程项目的不断完善与改进。同时, 特别致谢本次 PA 的设计者李世昌。

祝你在实现 Synera 的过程中, 既能完成验收目标, 也能真正提升面向对象设计与工程实践能力, PA 顺利!